

PR Audit Report

Repository	dipak-shaaki/demo-git-test-2
PR Number	#4
Risk Level	HIGH

PR Audit Report

Summary

This PR appears to add security-related utilities and CI support: a request logging middleware, helper functions, a checksum utility, a YAML config loader, and a workflow intended to trigger external security scanning on pull requests.

What the Developer Was Trying to Do

The developer seems to be:

- adding request logging middleware for observability,
- adding utility helpers for dictionary merging and debug output,
- adding a checksum helper,
- adding configuration loading utilities,
- and introducing a GitHub Actions workflow to trigger security scanning on PRs.

Security Issues (Bandit + Dependabot + SonarCloud)

1) Weak cryptographic hash: SHA1 used for checksum generation

File: app/utils/security.py

Function with issue:

```
import hashlib

def generate_checksum(data: str):
    return hashlib.sha1(data.encode()).hexdigest() # <- ISSUE HERE
```

Why this is problematic / risk:

- Bandit flags SHA1 as a weak hash for security-sensitive use.
- SHA1 is vulnerable to collision attacks and should not be used for security purposes such as integrity protection, signatures, or password-related workflows.
- If this checksum is used for security decisions, it could be bypassed or forged.

Corrected version of the full function:

```
import hashlib

def generate_checksum(data: str):
    return hashlib.sha256(data.encode()).hexdigest()
```

Official documentation:

- Python hashlib: <https://docs.python.org/3/library/hashlib.html>
 - Bandit SHA1 rule: https://bandit.readthedocs.io/en/1.9.4/plugins/b324_hashlib.html
-

2) Unsafe YAML loading can instantiate arbitrary objects

File: configs/settings.py

Function with issue:

```
import yaml

CONFIG_PATH = "configs/app_config.yaml"

def load_config():
    with open(CONFIG_PATH, "r") as f:
        config = yaml.load(f) # <- ISSUE HERE
```

```
return config
```

Why this is problematic / risk:

- `yaml.load()` without an explicit safe loader is unsafe.
- It can deserialize arbitrary Python objects, which may lead to code execution or other unsafe behavior if the YAML content is attacker-controlled.
- For configuration files, `yaml.safe_load()` is the appropriate choice.

Corrected version of the full function:

```
import yaml

CONFIG_PATH = "configs/app_config.yaml"

def load_config():
    with open(CONFIG_PATH, "r") as f:
        config = yaml.safe_load(f)

    return config
```

Official documentation:

- PyYAML documentation: <https://pyyaml.org/wiki/PyYAMLDocumentation>
- Bandit YAML load rule: https://bandit.readthedocs.io/en/1.9.4/plugins/b506_yaml_load.html

Code Quality Issues (Ruff + SonarCloud)

No Ruff findings were reported for the changed files.

1) Potential hard-coded credential detected

File: `app.py`

Finding from SonarCloud:

- Line 11
- Rule: `python:S2068`
- Message: "password" detected here, review this potentially hard-coded credential.

Why this is problematic / risk:

- SonarCloud is warning that a password-like value may be hard-coded in source.

- Hard-coded credentials can be leaked through source control, logs, or build artifacts and are difficult to rotate safely.

Corrective guidance:

- Move the credential to environment variables or a secrets manager.
- Ensure no secret values are committed to the repository.

Official documentation:

- Sonar rule: <https://rules.sonarsource.com/python/RSPEC-2068/>
-

2) Hard-coded PostgreSQL password detected

File: config/config.py

Finding from SonarCloud:

- Line 1
- Rule: secrets:S6698
- Message: Make sure this PostgreSQL password gets changed and removed from the code.

Why this is problematic / risk:

- A database password appears to be embedded in code.
- This creates a direct secret exposure risk and can lead to unauthorized database access if the repository is shared or compromised.

Corrective guidance:

- Remove the password from source code.
- Load it from environment variables or a secret store.
- Rotate the exposed credential if it has ever been committed.

Official documentation:

- Sonar rule: <https://rules.sonarsource.com/python/RSPEC-6698/>
-

3) Potential hard-coded secret detected

File: config/config.py

Finding from SonarCloud:

- Line 3
- Rule: python:S6418
- Message: "SECRET" detected here, make sure this is not a hard-coded secret.

Why this is problematic / risk:

- SonarCloud detected a likely secret value in code.
- Hard-coded secrets are a common source of credential leakage and should be externalized.

Corrective guidance:

- Replace the hard-coded value with a runtime secret source such as environment variables or a secrets manager.
- Review the file for any other embedded credentials.

Official documentation:

- Sonar rule: <https://rules.sonarsource.com/python/RSPEC-6418/>

Teammate Comments Summary

No existing teammate review comments were provided.

Overall Risk Assessment

- **Risk Level:** HIGH
- **Reason:** The PR introduces at least one unsafe YAML deserialization issue and one weak cryptographic hash, and SonarCloud also reports multiple hard-coded secret/credential concerns in other files.